

Empirical Analysis of Logistic Regression: Optimization, Convergence, and Linear Limitations

Benjamin Öztürk, Téo Roccamatysi

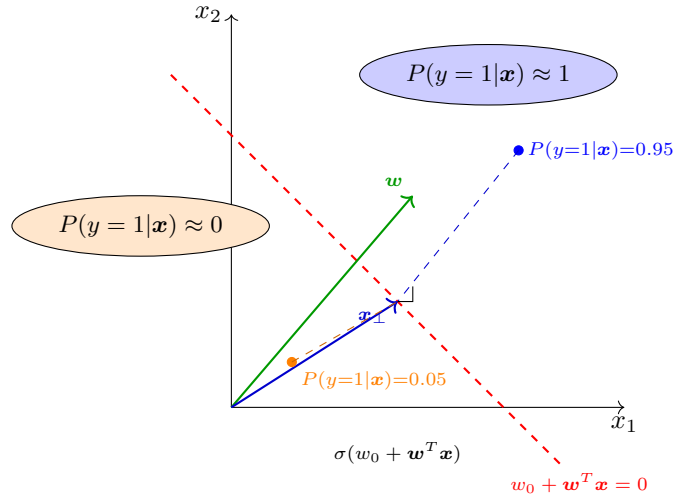
May 2025

1 Introduction

The objective of this practical work is to implement a **binary classification** model to separate two classes of points (red and green) using a **logistic regression**. Unlike classical regression methods that predict a continuous value, logistic regression is a **probabilistic model** that estimates the probability of a point belonging to a class (for example, the green class) based on its features.

The principle relies on a **sigmoid function**, which transforms a linear combination of input variables into a value between **0 and 1**, interpretable as a probability. A classification decision is then made by comparing this probability to a **critical threshold**, generally set at **0.5**:

- If $P(\text{class} = 1 \mid \mathbf{x}) \geq 0.5$, the point is assigned to class 1.
- Otherwise, it is classified into class 0.



Principle of Logistic Regression. The diagram above illustrates the mechanics of **binary logistic regression** in a two-dimensional space (x_1, x_2) . The model's objective is to predict the probability that the observation $\mathbf{x} = (x_1, x_2)$ belongs to class $y = 1$, denoted as $P(y = 1 \mid \mathbf{x})$.

The **decision boundary** is a line (dashed red) with the equation $w_0 + \mathbf{w}^T \mathbf{x} = 0$, where $\mathbf{w}$ is a weight vector (shown in green) and w_0 is a bias. This vector $\mathbf{w}$ is *orthogonal* to the decision boundary and defines the direction along which the logistic function $\sigma(z) = \frac{1}{1+e^{-z}}$ is applied.

Two sample points are shown:

- A blue point $(\mathbf{x}_1)$, located on the right side of the boundary, for which $P(y = 1 \mid \mathbf{x}_1) \approx 0.95$.
- An orange point $(\mathbf{x}_0)$, located on the left side, for which $P(y = 1 \mid \mathbf{x}_0) \approx 0.05$.

For each point, an **orthogonal projection** is drawn to the boundary. These projections allow identifying the value of the dot product $\mathbf{w}^T \mathbf{x}$, which represents the signed distance from the point to the hyperplane in the direction of $\mathbf{w}$. This value is then fed into the sigmoid function $\sigma(w_0 + \mathbf{w}^T \mathbf{x})$ to yield a **classification probability** between 0 and 1.

The blue and orange shaded zones indicate the regions where the estimated probability is close to 1 (class $y = 1$) or close to 0 (class $y = 0$), respectively. The model thus learns to separate the two classes probabilistically using a linear boundary.

2 Multiclass Logistic Regression Model

Logistic regression models the conditional probability that an event $y = 1$ occurs given an observation $\mathbf{x} \in \mathbb{R}^d$, using the sigmoid function:

$$\hat{y} = \sigma(\mathbf{w}^T \mathbf{x} + b) = \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{x} - b)} \quad (1)$$

where $\hat{y} = \mathbb{P}(y = 1 \mid \mathbf{x})$, $\mathbf{w} \in \mathbb{R}^d$ is a weight vector, and $b \in \mathbb{R}$ is a bias. The goal is to adjust these parameters so that the estimated probabilities are close to the observed classes.

The model can be generalized to the case where an observation $\mathbf{x} \in \mathbb{R}^d$ belongs to one of K possible classes. We then introduce a parameter vector $(\mathbf{w}_k, b_k)$ for each class $k \in \{1, \dots, K-1\}$, considering the K -th class as the reference. We model the likelihood ratios between classes using the following equations:

$$\begin{aligned} \log \left(\frac{\mathbb{P}(c = 1 \mid \mathbf{x})}{\mathbb{P}(c = K \mid \mathbf{x})} \right) &= \mathbf{w}_1^T \mathbf{x} + b_1 \\ \log \left(\frac{\mathbb{P}(c = 2 \mid \mathbf{x})}{\mathbb{P}(c = K \mid \mathbf{x})} \right) &= \mathbf{w}_2^T \mathbf{x} + b_2 \\ &\vdots \\ \log \left(\frac{\mathbb{P}(c = K-1 \mid \mathbf{x})}{\mathbb{P}(c = K \mid \mathbf{x})} \right) &= \mathbf{w}_{K-1}^T \mathbf{x} + b_{K-1} \end{aligned}$$

We deduce the probabilities for each class $k \in \{1, \dots, K\}$:

$$\mathbb{P}(c = k \mid \mathbf{x}) = \begin{cases} \frac{\exp(\mathbf{w}_k^T \mathbf{x} + b_k)}{1 + \sum_{\ell=1}^{K-1} \exp(\mathbf{w}_\ell^T \mathbf{x} + b_\ell)} & \text{if } k < K \\ \frac{1}{1 + \sum_{\ell=1}^{K-1} \exp(\mathbf{w}_\ell^T \mathbf{x} + b_\ell)} & \text{if } k = K \end{cases}$$

The model then returns a vector of probabilities over the K classes, the sum of which equals 1.

3 Likelihood and Loss Function

We assume that the observations are independent. Let $D = \{(\mathbf{x}_i, c_i)\}_{i=1}^N$ be a training dataset. The likelihood of the model is given by the product of the predicted probabilities for each observation:

$$\mathcal{L}(\boldsymbol{\theta}; D) = \prod_{i=1}^N \mathbb{P}(c_i \mid \mathbf{x}_i; \boldsymbol{\theta}) \quad (2)$$

In practice, we aim to maximize this likelihood, which is equivalent to minimizing the negative log-likelihood (loss function):

$$\mathcal{L}(\boldsymbol{\theta}; D) = - \sum_{i=1}^N \log \mathbb{P}(c_i \mid \mathbf{x}_i; \boldsymbol{\theta}) \quad (3)$$

This function measures how far the predicted probabilities deviate from the true observed classes. The lower the predicted probability for the correct class, the higher the penalty.

4 Optimization via Gradient Descent

In the binary case, the loss function can be explicitly written as:

$$\mathcal{L}(\mathbf{w}, b) = - \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (4)$$

where $\hat{y}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i + b)$ is the predicted probability.

The gradient of this function with respect to the weights is given by:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = X^\top (\hat{\mathbf{y}} - \mathbf{y}) \quad \text{and} \quad \frac{\partial \mathcal{L}}{\partial b} = \sum_{i=1}^N (\hat{y}_i - y_i) \quad (5)$$

where $X \in \mathbb{R}^{N \times d}$ is the data matrix, $\hat{\mathbf{y}}$ is the model output vector, and $\mathbf{y}$ is the vector of true classes. The weights are updated via gradient descent according to the rule:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial \mathcal{L}}{\partial \mathbf{w}}, \quad b \leftarrow b - \alpha \frac{\partial \mathcal{L}}{\partial b} \quad (6)$$

where $\alpha > 0$ is the learning rate. This process is repeated until the model converges to a minimum of the loss function.

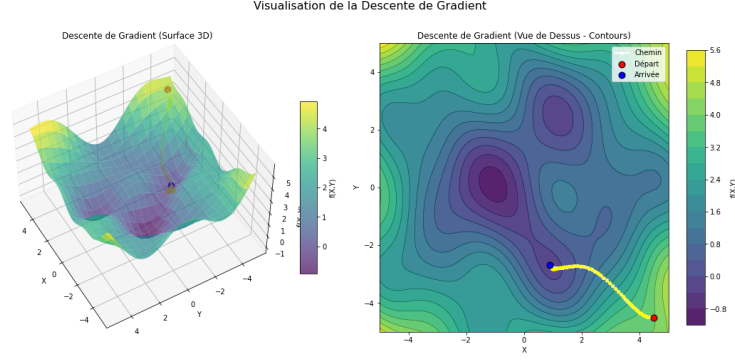


Figure 1: Gradient descent until reaching a minimum of the loss function

5 Practical Implementation on 3 Cases

5.1 Case 1:

We test the algorithm for different iterations with a fixed learning rate of $p = 0.01$.

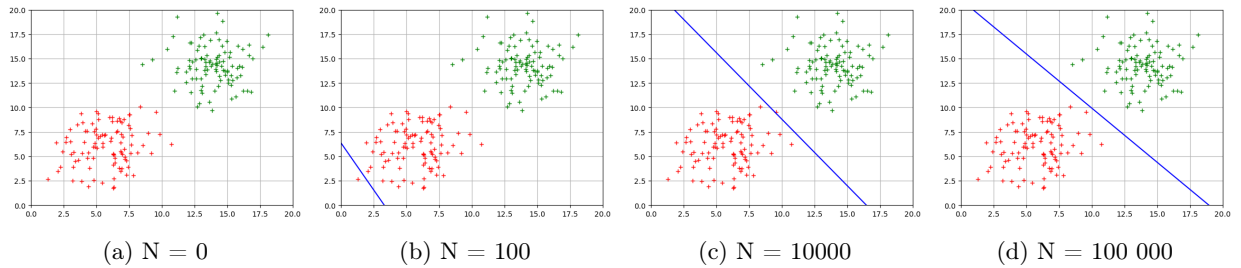


Figure 2: Decision boundary line after different numbers of iterations

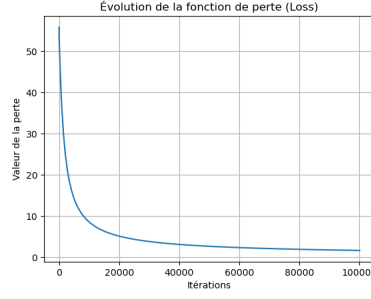


Figure 3: Loss function

It is observed that when the number of iterations N is insufficient, the loss function has not yet converged. This results in a poorly adjusted decision boundary and poor class separation. Conversely, when N is very high, the loss function becomes almost stable: the variations in $\frac{\partial \mathcal{L}}{\partial N}$ become negligible. A compromise between performance and computation time must therefore be chosen, ensuring that the model has had enough time to converge.

5.2 Case 2:

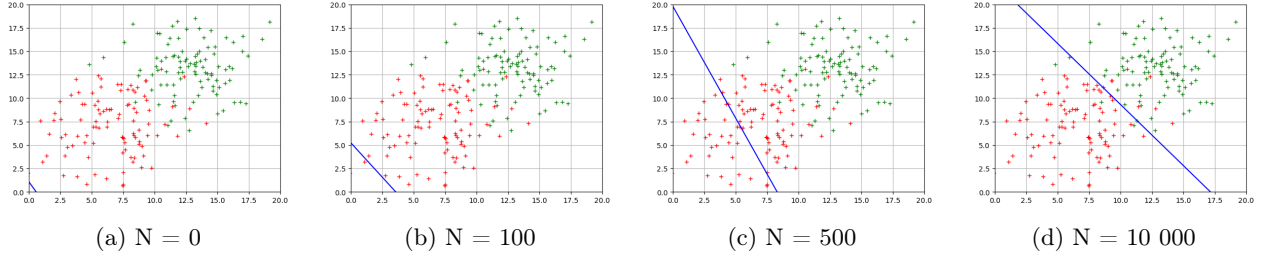


Figure 4: Effect of the number of iterations N (with $p = 0.01$)

As seen previously, we notice that an insufficient number of iterations does not allow for a clear separation of our two categories.

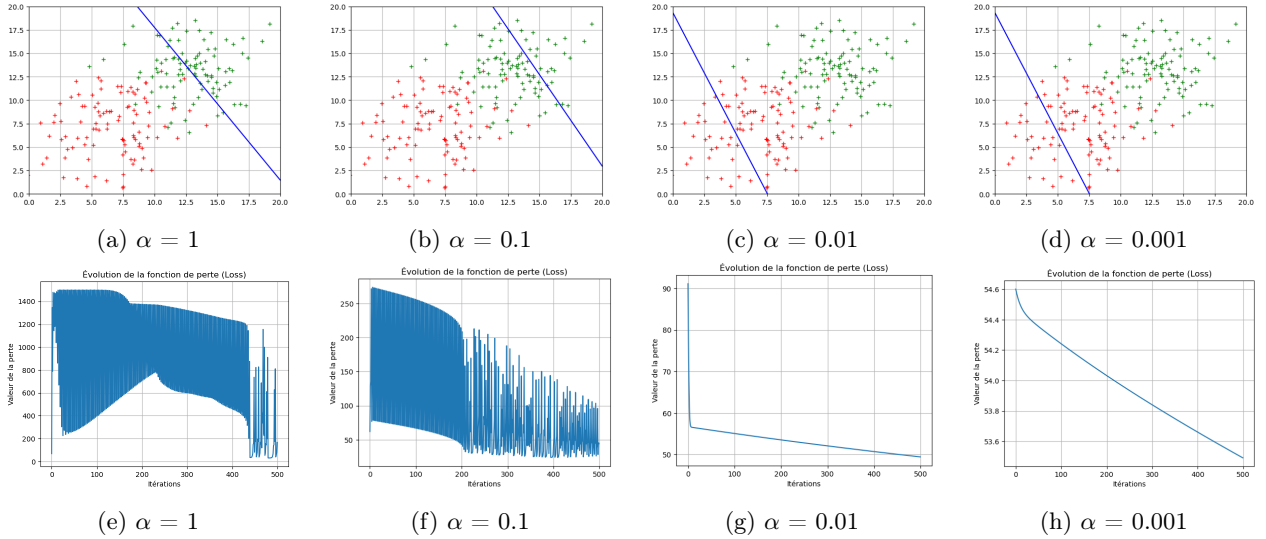


Figure 5: Effect of the learning rate α on the loss function (with $N = 500$).

When α is too high, instability in the loss function is observed: it oscillates or even diverges. The function $\mathcal{N} \rightarrow \mathcal{L}(\mathcal{N})$ no longer strictly decreases. This indicates that the update steps are too large, causing

the algorithm to “jump” instead of descending smoothly. Conversely, an α that is too low slows down convergence: the model learns, but very slowly. A proper choice of α is therefore crucial to ensure fast and stable convergence.

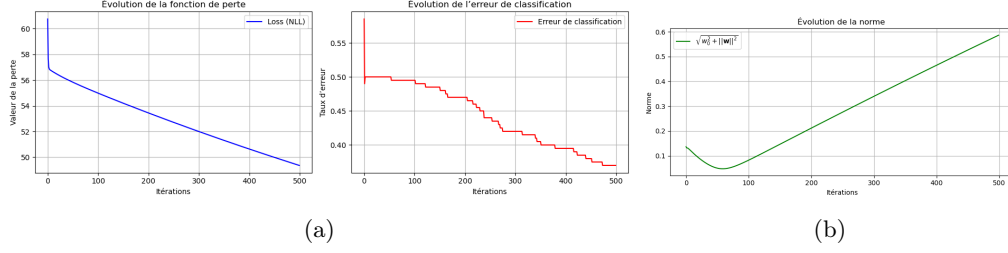


Figure 6: Evolution of different parameters $N = 500$; $\alpha = 0.01$ (a) Evolution of the loss function and error as a function of the number of iterations; (b) $\sqrt{w_0^2 + ||w||^2}$ as a function of the number of iterations

5.3 Case 3:

In Case 3, we have a cluster of red points in the middle of a green area.

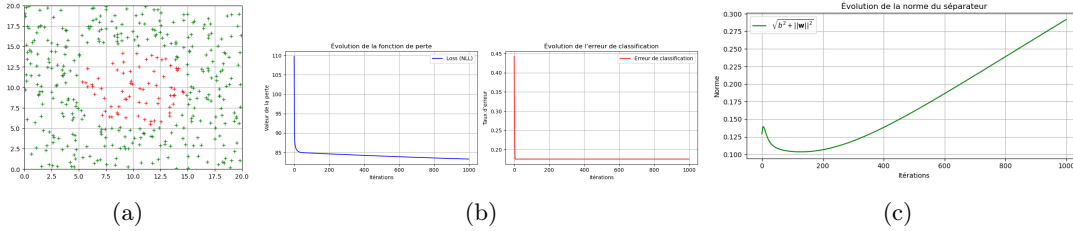


Figure 7: (a) Scatter plot; (b) Evolution of the loss function and error as a function of the number of iterations; (c) $\sqrt{w_0^2 + ||w||^2}$ as a function of the number of iterations

Param: $N = 500$; $\alpha = 0.01$

In the third case, we observe a cluster of red points located at the center of a group of green points forming a ring. This configuration is not linearly separable, meaning that no straight line (or hyperplane) exists that can effectively separate the two classes. However, logistic regression relies precisely on this assumption: it searches for a linear decision boundary, which corresponds to a line in the two-dimensional space considered here. Thus, regardless of the training quality, the model will fail to construct a coherent separation between the classes.

This behavior results in the loss function stagnating around a relatively high value, and an inability to significantly reduce the classification error, as observed here.

6 Conclusion

In this practical work, we implemented and analyzed logistic regression as a method for binary classification. Through three distinct cases, we studied the impact of the learning rate α and the number of iterations N on the convergence of the algorithm and the quality of the predictions.

We found that an poorly chosen learning rate can lead either to model divergence or to an excessively slow learning process. Similarly, an insufficient number of iterations prevents the convergence of the loss function. A judicious choice of these two hyperparameters is therefore essential for efficient training.

Finally, the last case highlighted the fundamental limitations of logistic regression: when a linear separation of the data is not possible, the model cannot learn an effective boundary, regardless of the optimization quality. This highlights the necessity of using non-linear models.

La classification binaire est une tâche élémentaire et importante du Machine Learning. Son objectif est souvent de prédire une propriété (ou non) pour une entrée donnée. Prenons un exemple. Nous disposons d'un jeu de données avec N composants chimiques qui sont classés selon qu'ils soient biodégradables ou non (le caractère biodégradable a été établi par les chimistes selon un protocole expérimental en laboratoire). Chacun de ces composants est décrit par un vecteur regroupant différentes caractéristiques numériques qui caractérisent diverses propriétés physico-chimiques.

- nombre d'atomes lourds
- pourcentage d'atome de carbone
- pourcentage d'atome de d'oxygène
- nombre de groupe nitro (aromatique)
- fréquence de N-N à une distance topologique de 1
- ...

Ainsi nous disposons d'un jeu de données : $D = (\mathbf{x}_i, c_i)$ avec $i = 1$ à N , et N est le nombre d'exemples d'apprentissage. Ainsi $\mathbf{x}_i$ est l'entrée du modèle de classification et c_i la "bonne réponse" ou dit autrement la réponse attendue.

L'objectif est d'apprendre à partir de D un modèle capable de prédire si un composant chimique représenté par un vecteur $\mathbf{x}$ est biodégradable $c = 1$ ou non $c = 0$. Dans ce cas, chaque vecteur $\mathbf{x}$ est un vecteur de taille 41. Ce jeu de données est décrit [ici](#). Apprendre veut dire: trouver un modèle capable de reproduire au mieux la prédiction de c_i à partir de $\mathbf{x}_i, \forall i$.

Un autre exemple en Physique, pour trier les données obtenues par le CERN lors des travaux sur le Boson de Higgs, la classification binaire a été utilisée. Par exemple, chaque événement est représenté par un vecteur de 30 caractéristiques décrivant les propriétés cinétiques des particules détectées, des mesures d'énergie, informations spatiales et inertielles, L'objectif est de prédire si l'événement correspond à un signal (Boson) ou non.

La régression logistique permet de faire de la classification binaire. C'est un modèle qui définit une séparation linéaire (un hyperplan) dans l'espace d'entrée (l'espace des $\mathbf{x}$) et qui calcul une probabilité d'appartenir à la classe 1 en fonction de la distance du point à l'hyperplan. Commençons par visualiser cette notion. Pour cela nous allons travailler sur des données synthétiques en 2 dimensions ($\mathbf{x} \in \mathbb{R}^2$) afin de visualiser ces notions.

Chargement des données

Le premier jeu de données est stocké dans le fichier `reglog_data_1.mat`. Le chargement s'effectue ainsi.

```
In [7]: # Quelques imports d'abord
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
from scipy.io import loadmat # module to load matlab data
```

```
In [8]: # Ici on suppose que le fichier reglog_data_1.mat est présent dans le même répertoire.
# Sinon, il faut ajouter le chemin au nom du fichier.
dataset = loadmat("reglog_data_1.mat")
X = dataset["X"] # l'ensemble des vecteur x
C = dataset["C"] # les classes associées à chaque x
print(X.shape, C.shape)
```

```
(2, 200) (1, 200)
```

```
In [9]: print(X[:,153], C[:,153])
```

```
[12.15585597 13.99791643] [1]
```

La fonction suivante est donnée et permet de visualiser les données

```
In [10]: def plotdata(X, C, w=None, b=None):
    """
    Fonction qui représente
    - les données X (une matrice 2,N avec N le nombre de points),
    - associées à leurs classes (binaires) données par C (1,N)
    - w et b sont optionnels et permettent de tracer la droite séparatrice tel que w.x + b = 0
    Les données sont supposées être entre 0 et 20.
    """
    MIN = 0
    MAX = 20
    pos = (C==1).squeeze()
    neg = (C==0).squeeze()
    plt.plot(X[0,pos], X[1,pos], "g+")
    plt.plot(X[0,neg], X[1,neg], "r+")
    if w is None or b is None:
        plt.xlim(MIN, MAX)
        plt.ylim(MIN, MAX)
        plt.grid()
        plt.show()
```

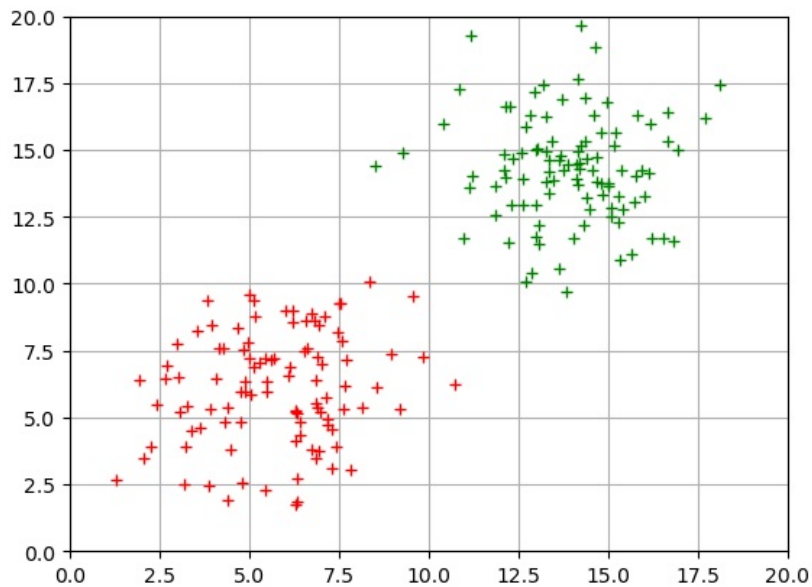
```

    return
    if (w[1]==0):
        if (w[0] == 0):
            print("WARNING cannot plot such line ...")
            return
        x1=-b/w[0]
        y1=MAX
        x2=x1
        y2=MIN
    else:
        x1=MAX
        y1=-(b+w[0]*x1)/w[1]
        x2=MIN
        y2=-(b+w[0]*x2)/w[1]
    plt.plot([ x1 , x2 ], [ y1 , y2 ], "b-")
    plt.xlim(MIN,MAX)
    plt.ylim(MIN,MAX)
    plt.grid()
    plt.show()

```

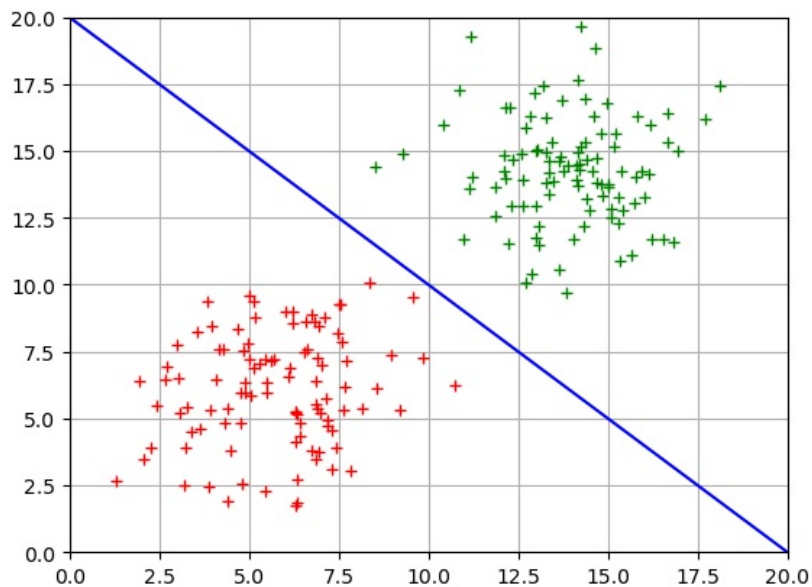
Essayons sur les données:

In [11]: `plotdata(X,C)`



On peut aussi dessiner une droite:

In [12]: `plotdata(X,C, w = np.ones(2), b = -20)`



Il existe de nombreux modèles et algorithmes pour trouver la droite séparatrice étant donnés `X` et `C`. Un des premiers est le perceptron (l'ancêtre des réseaux de neurones artificiels) mais dans ce TP nous allons voir un des plus utilisés: la régression logistique (un autre

ancêtre les réseaux de neurones artificiels).

Régression Logistique

La régression logistique permet de faire de la classification binaire. On considère que la classe associée à un élément $\mathbf{x}$ est une variable aléatoire binaire. L'objectif est de pouvoir prédire la probabilité de la classe 1 à partir $\mathbf{x}$: $P(c = 1 | \mathbf{x})$

Pour cela, le modèle s'appuie sur une séparation linéaire dans l'espace d'entrée par une hyperplan. De manière générale cet hyperplan a pour équation $\mathbf{w}'\mathbf{x} + b = 0$ pour tous les points $\mathbf{x}$ appartenant à l'hyperplan. Les paramètres qui définissent l'hyperplan sont $\mathbf{w}$ et b . $\mathbf{w}$ est un vecteur de même taille que $\mathbf{x}$. En dimension 2, un hyperplan est une droite.

Cet hyperplan sépare ainsi l'espace en partie (séparation binaire). La quantité $\mathbf{w}'\mathbf{x} + b$ nous donne les informations suivantes:

- son signe nous dit de quel côté de l'hyperplan se situe $\mathbf{x}$.
- sa valeur est proportionnelle à la distance entre $\mathbf{x}$ et l'hyperplan.

La régression logistique exploite cette quantité pour estimer la probabilité de l'événement binaire: le vecteur $\mathbf{x}$ appartient à la classe 1 (oui ou non). Dans ce TP nous allons travailler sur des données en 2D mais ce modèle se généralise à des dimensions quelconques.

Le modèle de régression logistique

Soit $\mathbf{x}$ un vecteur de $\mathbb{R}^d$ (avec $d = 2$ par exemple). La régression logistique calcul la probabilité de la classe $c = 1$ comme ceci:

$$P(c = 1 | \mathbf{x}) = \sigma(\mathbf{w}'\mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}'\mathbf{x} + b)}}$$

La fonction σ est la fonction logistique, aussi connue sous le nom de sigmoid. Elle permet de "convertir" la distance entre le point et l'hyperplan en une probabilité: si $\mathbf{w}'\mathbf{x} + b > 0$, alors $P(c = 1 | \mathbf{x}) > 0.5$ et on peut considérer que le modèle prédit classe 1 pour $\mathbf{x}$. Au contraire, si $\mathbf{w}'\mathbf{x} + b < 0$, alors $P(c = 1 | \mathbf{x}) < 0.5$ et on peut considérer que le modèle prédit classe 0 pour $\mathbf{x}$.

Rappelons que pour la fonction sigmoid σ et $a \in \mathbb{R}$:

$$\sigma(a) = \frac{1}{1 + e^{-a}}, \quad \sigma'(a) = \sigma(a)(1 - \sigma(a))$$

Fonction de perte

Les paramètres du modèle, $\boldsymbol{\theta} = (\mathbf{w}, b)$ et la question est maintenant de comment trouver les valeurs de ces paramètres ? En apprentissage machine, une méthode consiste à définir une fonction dite de perte (ou *loss*) qui caractérise les défauts de $\boldsymbol{\theta}$, l'objectif est alors de la minimiser. La fonction de perte est pour la régression logistique la **négative log-vraisemblance** (ou *negative-loglikelihood*, ou encore NLL). Elle se définit ainsi:

$$\mathcal{L}(\boldsymbol{\theta}; D) = - \sum_{i=1}^N \log P(c_{(i)} | \mathbf{x}_{(i)}) = - \sum_{i=1}^N \left[c_{(i)} \log P(c_{(i)} | \mathbf{x}_{(i)}) + (1 - c_{(i)}) \log (1 - P(c_{(i)} | \mathbf{x}_{(i)})) \right]$$

Minimiser cette fonction est équivalent à maximiser la vraisemblance, i.e la probabilité que donne le modèle paramétré par $\boldsymbol{\theta}$ au jeu de données D .

Optimisation

Un algorithme de minimisation est la descente de gradient. Il se résume ainsi:

- initialisation aléatoire des paramètres $\boldsymbol{\theta}$.
- calcul du gradient $\mathbf{g}$ de $\mathcal{L}(\boldsymbol{\theta}; D)$ par rapport à $\boldsymbol{\theta}$.
- Mise à jour des paramètres:

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \alpha \mathbf{g},$$

avec α un hyper-paramètre de l'algorithme nommé le pas d'apprentissage (ou *learning rate*).

Objectifs du TP

Coder et expérimenter la descente de gradient afin d'apprendre un modèle de régression logistique. Les expériences seront menées sur trois jeu de données consécutifs.

Les fonctions principales

Pour apprendre ce modèle, nous avons besoin des fonctions suivante:

- Les paramètres sont initialisés aléatoirement comme des réalisations de variables gaussiennes centrées réduites. Le code est fourni. Les paramètres pourraient être initialisés autrement, par les moindres carrés ordinaires par exemple.
- inférence: à partir de $\mathbf{x}$ calcul $y = P(c | \mathbf{x})$ (cette fonction peut prendre un vecteur $\mathbf{x}$ ou un ensemble de vecteur (une matrice) et ainsi retourner l'ensemble des probabilités (un vecteur).
- Calcul de la fonction objectif
- Calcul du gradient de la fonction objectif vis-à-vis des paramètres. Pour cette fonction nous devons connaître la formule analytique du gradient.

TODO: La première fonction est déjà écrite. A vous de compléter les suivantes.

```
In [13]: def init_weights(std=0.1):
    """Initialize weights with a gaussian of 0 mean
    Args: @
        - std: the variance
    Returns:
        - w is a np.array of shape (1,2)
        - b is a float
    """
    w = np.random.randn(1,2)*std
    b = np.random.randn()*std
    return w,b

def inference(x,weights,bias):
    """
    Returns the probability to be in the class 1
    The previous formula is interpreted as math !
    output: np.array with one probability per input points

    Args:
        - x is a np.array of shape (2,N) with N the number of input points.
        - weights : the weight vector, numpy array of shape (1,2)
        - bias : the bias term, a float

    """
    z = weights @ x + bias
    return 1 / (1 + np.exp(-z))

def loss_fn(predictions, goodanswers):
    """
    Compute the negative-log-likelihood of predictions (a set of predictions),
    given that goodanswers gathers the good answers (gold standard).
    The predictions can be computed by the function inference on a subpart or all the examples in X.
    goodanswers can be subpart of all the C
    Args:
        - both are numpy arrays of the same shape (N).

    Returns:
        The loss function value for all the inputs (scalar)
    """
    p = predictions
    g = goodanswers
    epsilon = 1e-15
    return - np.sum(g * np.log10(p + epsilon) + (1-g)*np.log10(1-p + epsilon))

def parameters_gradient_loss(goodanswer, predictions, inputs):
    """
    Compute the gradients of the loss function (negative-log-likelihood)
    Args:
        - goodanswers : the expected answers (numpy array of size 1,N)
        - predictions : the predictions or the answer of the model (numpy array of size 1,N)
        - x : the inputs (numpy array of shape (2,N))

    Returns:
        The gradient w.r.t the weights and the bias term:
        - dw : a numpy array of the shape (1,2)
        - db : a numpy array of shape (1)
    """
    g = goodanswer
    p = predictions
    x = inputs
    m = x.shape[1]
    error = p - g
    dw = (1 / m) * np.dot(error, x.T)
    db = (1 / m) * np.sum(error)
    return dw, db
```

```
In [14]: # Test each functions before the next step
w,b = init_weights(std=0.1)
```

```
I = inference(X,w,b)
print(I)
```

```
[[0.32946423 0.3886002 0.37741103 0.35118193 0.33200695 0.34376794
0.35918478 0.34321744 0.42627576 0.26672588 0.33360045 0.25849839
0.41775115 0.31845085 0.38453252 0.30735338 0.33700789 0.2996087
0.31116833 0.34091941 0.33818331 0.3303334 0.37708921 0.38913255
0.35264335 0.35621824 0.37360462 0.38983622 0.36067457 0.33922358
0.39441969 0.33313245 0.32898939 0.30012733 0.41403161 0.32782927
0.3456823 0.33253916 0.29780521 0.31440472 0.37198984 0.30844917
0.37680506 0.32909228 0.33564253 0.3287714 0.38744392 0.34952162
0.30556113 0.31208172 0.35801939 0.27929354 0.35929494 0.30587153
0.34563499 0.31613827 0.34081069 0.39427457 0.38977609 0.31758546
0.31437143 0.37003811 0.30325254 0.38906917 0.34772813 0.32120697
0.28818316 0.35976079 0.41367178 0.44765109 0.36056574 0.35521121
0.33796775 0.34652638 0.34160857 0.37980011 0.29500457 0.3574461
0.28448642 0.42310797 0.35472877 0.32493922 0.33767307 0.34626963
0.30851888 0.3375636 0.39544707 0.34676988 0.35599992 0.33745589
0.37290934 0.30323066 0.39941917 0.29719428 0.28487323 0.40411286
0.30054172 0.28001808 0.31612673 0.34833344 0.17168185 0.13255492
0.17426129 0.19867051 0.19976878 0.15890547 0.18432215 0.19242764
0.19073464 0.16428134 0.15685075 0.13680251 0.17556453 0.16463865
0.17269039 0.22219548 0.15631403 0.15626417 0.18092151 0.15793934
0.18313029 0.14762117 0.16968589 0.16235657 0.16711071 0.18423944
0.16418712 0.14443566 0.17820228 0.16912675 0.18101056 0.18572691
0.17085357 0.14532015 0.18811633 0.17624566 0.1227683 0.1699517
0.16777032 0.16453055 0.1697515 0.17119713 0.17629561 0.21580153
0.1983644 0.17774061 0.14539035 0.16227589 0.19683018 0.20442312
0.17350835 0.13807783 0.20261799 0.19140327 0.18720366 0.18459437
0.1718486 0.22048275 0.17509345 0.18007295 0.16692258 0.21034511
0.17943573 0.17480648 0.17533527 0.16095483 0.14497707 0.16832399
0.15378005 0.15371693 0.17637172 0.17529701 0.17637775 0.17202884
0.17367036 0.17028595 0.17076121 0.1343857 0.16922368 0.20617974
0.22244723 0.17200946 0.16039739 0.15766896 0.20374994 0.15094322
0.19586825 0.16968795 0.21291076 0.19329223 0.17312727 0.18974447
0.17058869 0.1576969 0.14763699 0.18849989 0.17107095 0.17821236
0.21174869 0.20718632]]
```

```
In [15]: print(loss_fn(I, C))
```

94.43724232504057

```
In [16]: print(parameters_gradient_loss(C, I, X))
```

(array([[-4.79568615, -4.89318274]]), -0.24047854864132837)

Premières expériences

Maintenant que nous avons toutes les étapes nécessaires, nous pouvons expérimenter l'algorithme d'apprentissage sur les données. Le nombre d'itérations d'apprentissage (le passage sur toutes les données) est une variable de l'algorithme.

TODO:

1. Mettre en place la boucle d'apprentissage et tracer l'évolution de la fonction de coût au cours de l'apprentissage. Pour commencer, fixer le pas d'apprentissage à 0.01 et lancer 100 itérations.
2. Il peut être plus simple pour l'interprétation de tracer le taux d'erreur de classification.
3. Visualiser la frontière de décision et mettre en relation avec les grandeurs précédentes.

Que faire pour améliorer le résultat ?

```
In [17]: # Paramètres
pas = 0.01
N = 1000

# Initialisation des poids
w, b = init_weights(std=0.1)
plotdata(X, C,w[0], b )
# Historique des pertes
losses = []

# Boucle d'entraînement
for i in range(N):
    predictions = inference(X, w, b)
    loss = loss_fn(predictions, C)
    losses.append(loss)

    dw, db = parameters_gradient_loss(C, predictions, X)

    w = w - pas * dw
    b = b - pas * db
```

```
# Tracé des données et de la frontière
```

```
plotdata(X, C, w.flatten(), b)
```

```
# Tracé de l'évolution de la loss
```

```
plt.plot(losses)
```

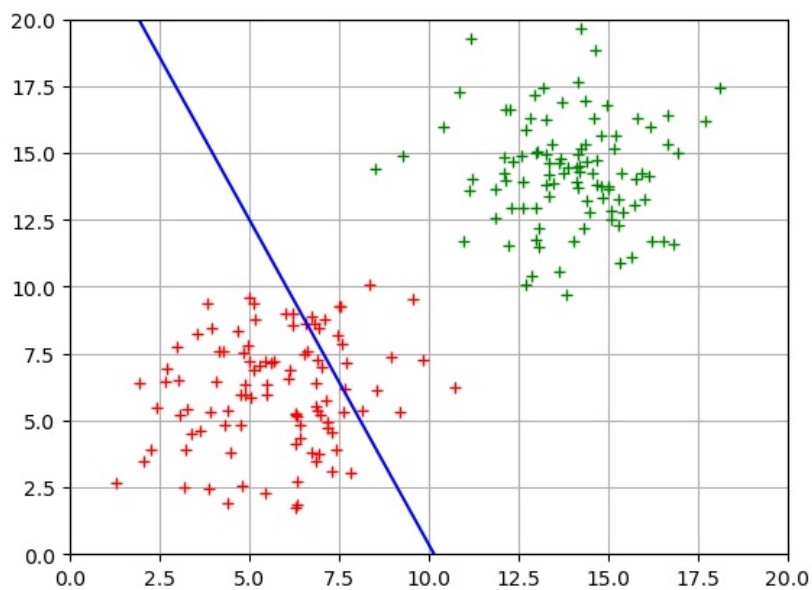
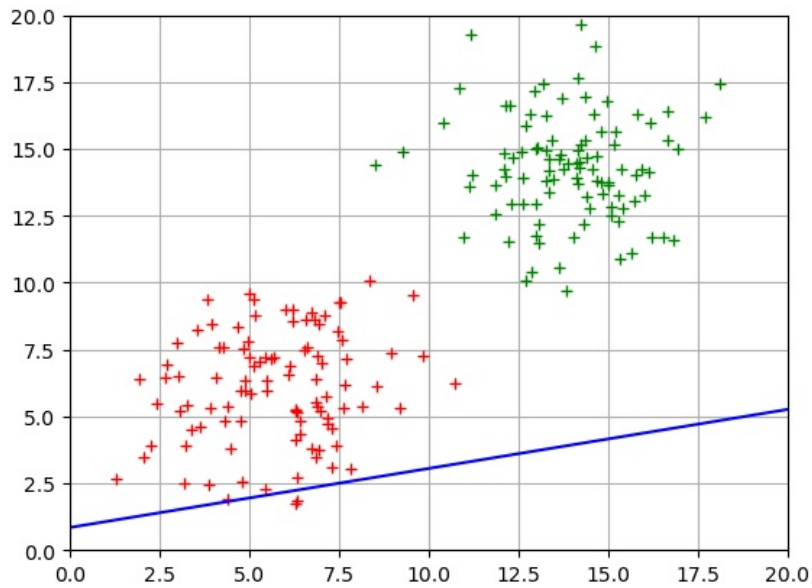
```
plt.title("Évolution de la fonction de perte (Loss)")
```

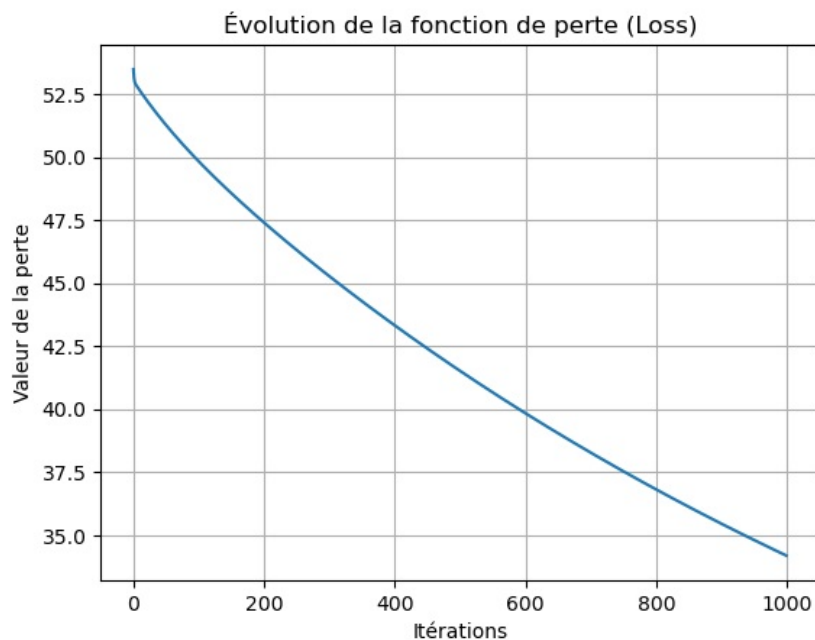
```
plt.xlabel("Itérations")
```

```
plt.ylabel("Valeur de la perte")
```

```
plt.grid()
```

```
plt.show()
```





Un cas plus difficile

Considérons désormais les données du fichier `reglog_data_2.mat` et commençons par les visualiser.

1. Mettre en place la boucle d'apprentissage avec un pas d'apprentissage de 0.01 et 500 itérations,
2. Tracer l'évolution de la fonction de coût au cours de l'apprentissage ainsi que celle du taux d'erreur de classification.
3. Augmenter le pas d'apprentissage : essayer 0.1 puis 0.5
4. Revenons à la valeur du pas d'apprentissage de 0.01. Lancer l'apprentissage sur 1000 itérations. Tracer également l'évolution de la norme du vecteur des paramètres:

$$\sqrt{w_0^2 + \|\mathbf{w}\|^2}$$

```
In [18]: dataset = loadmat("reglog_data_2.mat")
X = dataset["X"] # l'ensemble des vecteur x
C = dataset["C"] # les classes associées à chaque x
print(X.shape, C.shape)
```

(2, 200) (1, 200)

```
In [26]: # Paramètres
pas = 0.01
N = 500

# Initialisation des poids
w, b = init_weights(std=0.1)
plotdata(X, C, w[0], b)
# Historique des pertes
losses = []

# Boucle d'entraînement
```

```

for i in range(N):
    predictions = inference(X, w, b)
    loss = loss_fn(predictions, C)
    losses.append(loss)

    dw, db = parameters_gradient_loss(C, predictions, X)

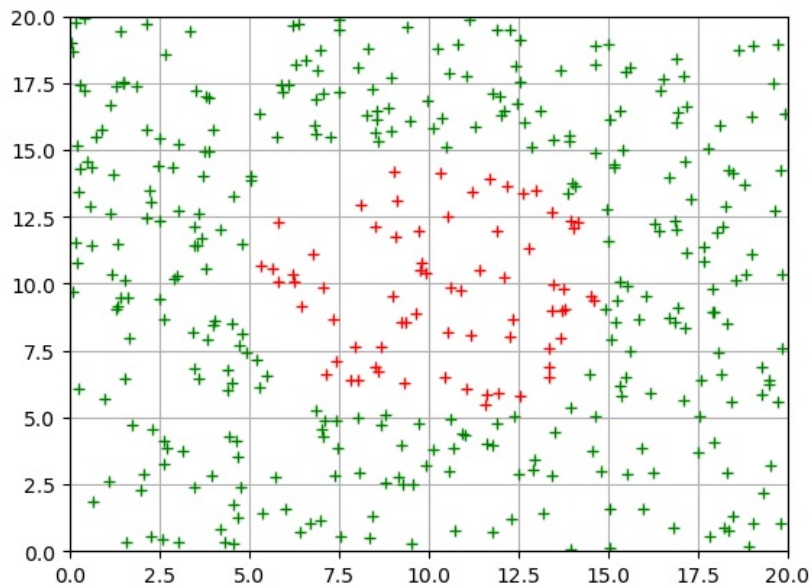
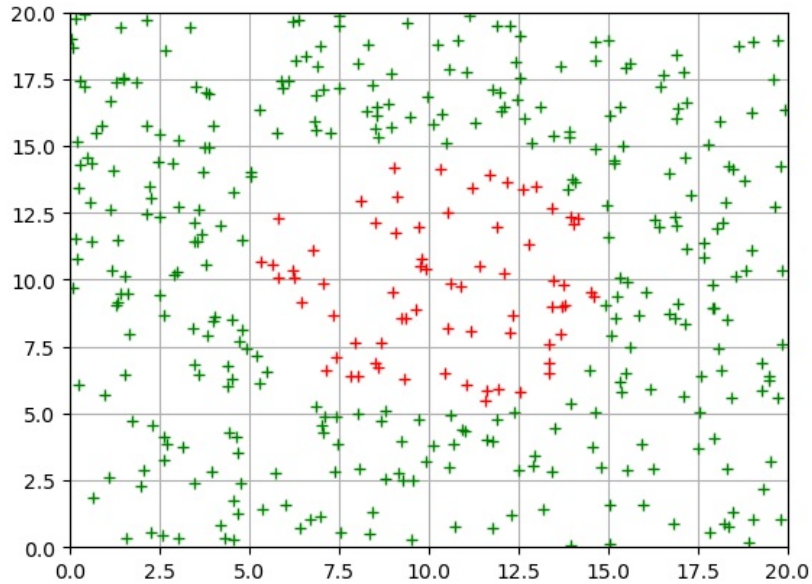
    w = w - pas * dw
    b = b - pas * db

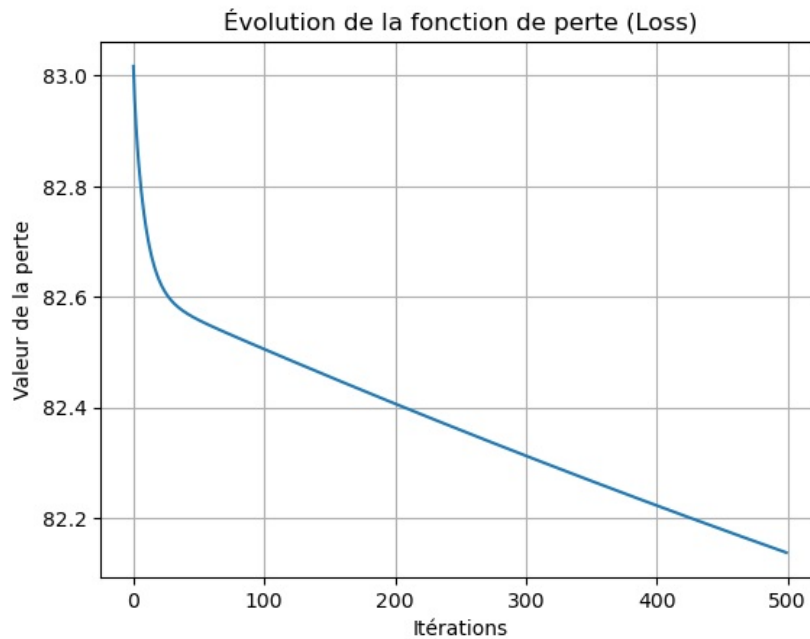
# Tracé des données et de la frontière

plotdata(X, C, w.flatten(), b)

plt.plot(losses)
plt.title("Évolution de la fonction de perte (Loss)")
plt.xlabel("Itérations")
plt.ylabel("Valeur de la perte")
plt.grid()
plt.show()

```





```
In [21]: pas = 0.01
N = 10000

# Initialisation des poids
w, b = init_weights(std=0.1)

# Historique
losses = []
erreur = []
normes = []

# Boucle d'entraînement
for i in range(N):
    predictions = inference(X, w, b)

    loss = loss_fn(predictions, C)
    losses.append(loss)

    classes_predites = (predictions >= 0.5).astype(int)
    err = np.mean(classes_predites != C)
    erreur.append(err)

    norme = np.sqrt(b**2 + np.sum(w**2))
    normes.append(norme)

    dw, db = parameters_gradient_loss(C, predictions, X)
    w = w - pas * dw
    b = b - pas * db

# Figure 1 : frontière de décision
plotdata(X, C, w.flatten(), b)

# Figure 2 et 3 : loss et erreur côte à côte
fig, axs = plt.subplots(1, 2, figsize=(12, 4))

axs[0].plot(losses, label="Loss (NLL)", color='blue')
axs[0].set_title("Évolution de la fonction de perte")
axs[0].set_xlabel("Itérations")
axs[0].set_ylabel("Valeur de la perte")
axs[0].grid()
axs[0].legend()

axs[1].plot(erreur, label="Erreur de classification", color='red')
```

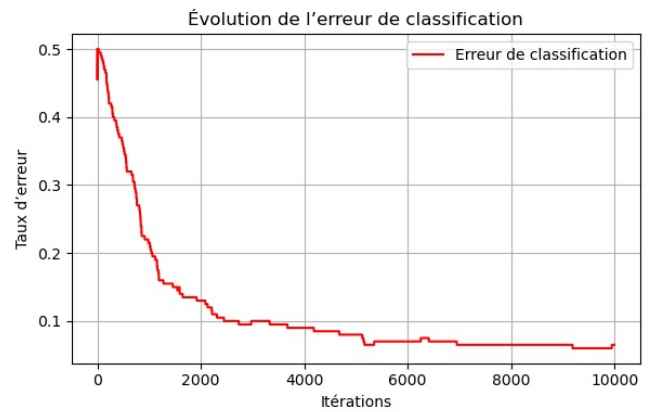
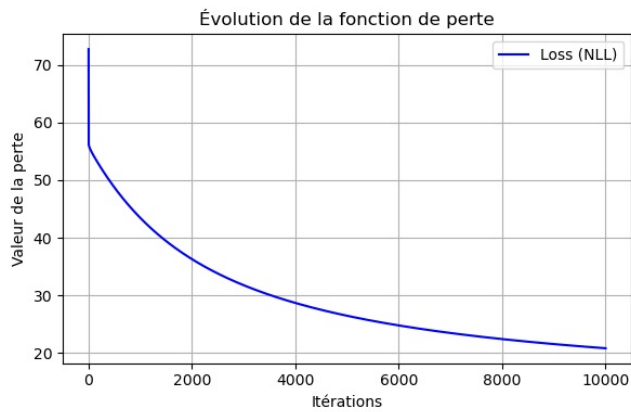
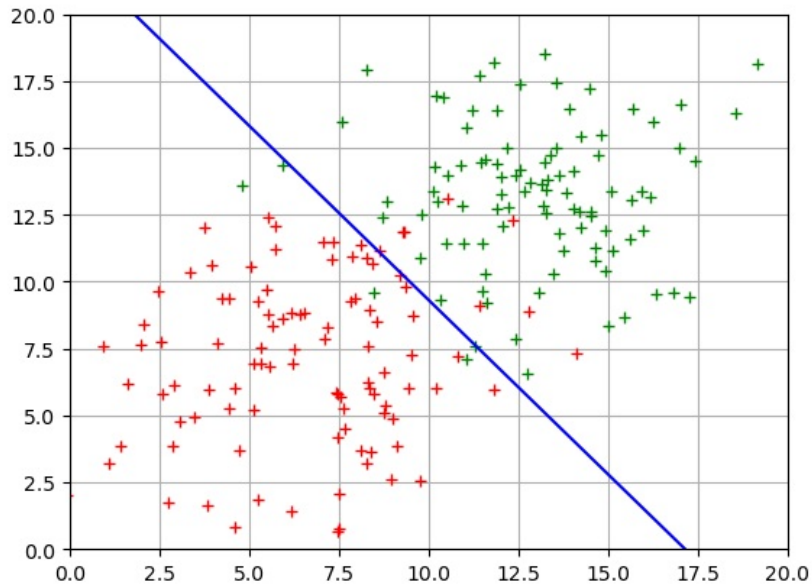
```

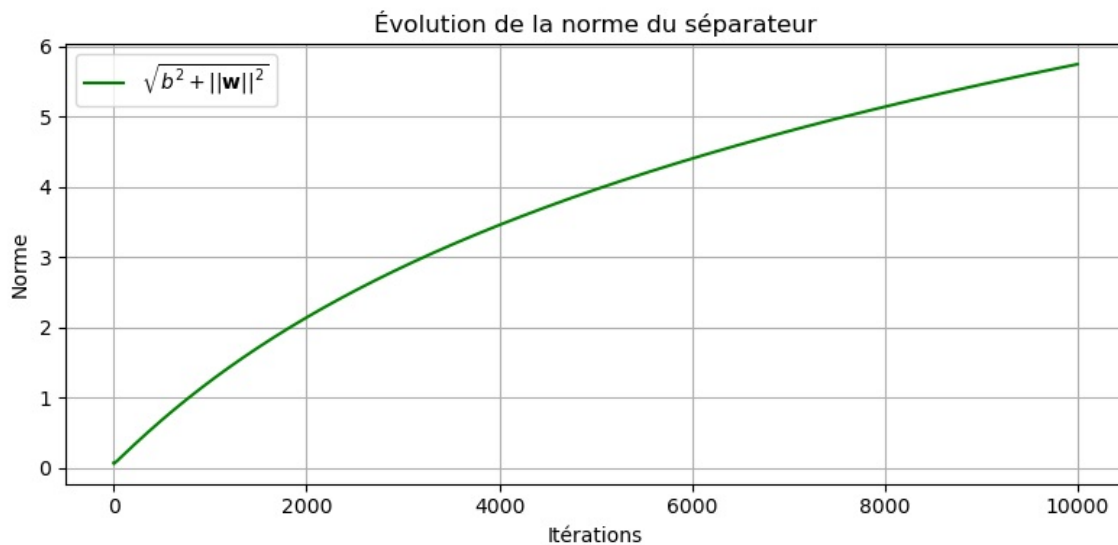
axs[1].set_title("Évolution de l'erreur de classification")
axs[1].set_xlabel("Itérations")
axs[1].set_ylabel("Taux d'erreur")
axs[1].grid()
axs[1].legend()

plt.tight_layout()
plt.show()

# Figure 4 : norme
plt.figure(figsize=(8, 4))
plt.plot(normes, label=r"$\sqrt{b^2 + ||\mathbf{w}||^2}$", color='green')
plt.title("Évolution de la norme du séparateur")
plt.xlabel("Itérations")
plt.ylabel("Norme")
plt.grid()
plt.legend()
plt.tight_layout()
plt.show()

```





Encore plus difficile

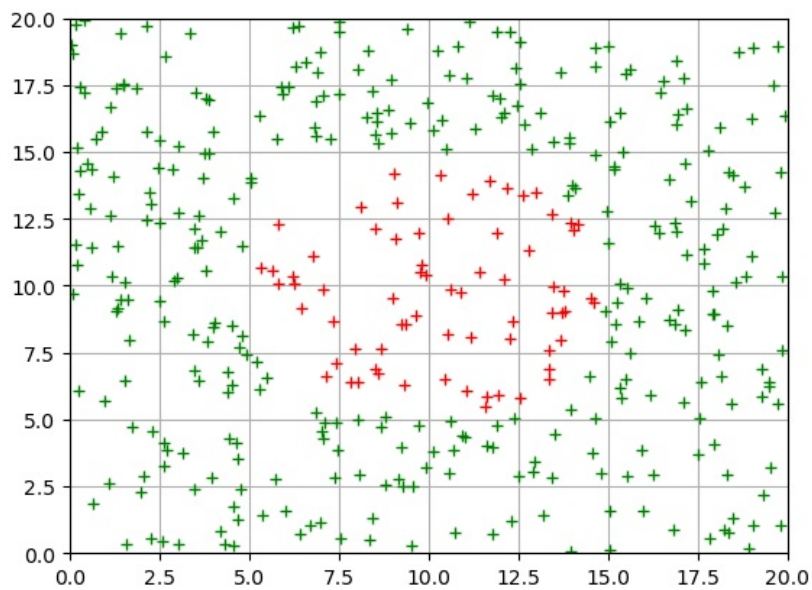
Considérons enfin les données du fichier `reglog_data_3.mat` et commençons par les visualiser.

1. Que donne l'apprentissage d'un modèle de régression logistique (pas d'apprentissage de 0.01 sur 1000 itérations) ? Interprétez le résultat.
2. Proposer une solution pour résoudre le problème.

```
In [23]: dataset = loadmat("reglog_data_3.mat")
X = dataset["X"]
C = dataset["C"]
print(X.shape, C.shape)
```

```
(2, 400) (1, 400)
```

```
In [24]: plotdata(X, C)
```

```
In [25]: pas = 0.01
N = 1000

# Initialisation des poids
w, b = init_weights(std=0.1)

# Historique
losses = []
erreur = []
normes = []

# Boucle d'entraînement
for i in range(N):
    predictions = inference(X, w, b)

    loss = loss_fn(predictions, C)
    losses.append(loss)

    classes_predites = (predictions >= 0.5).astype(int)
    err = np.mean(classes_predites != C)
    erreur.append(err)

    norme = np.sqrt(b**2 + np.sum(w**2))
    normes.append(norme)

    dw, db = parameters_gradient_loss(C, predictions, X)
    w = w - pas * dw
    b = b - pas * db

# Figure 1 : frontière de décision
plotdata(X, C, w.flatten(), b)

# Figure 2 et 3 : loss et erreur côte à côte
fig, axs = plt.subplots(1, 2, figsize=(12, 4))

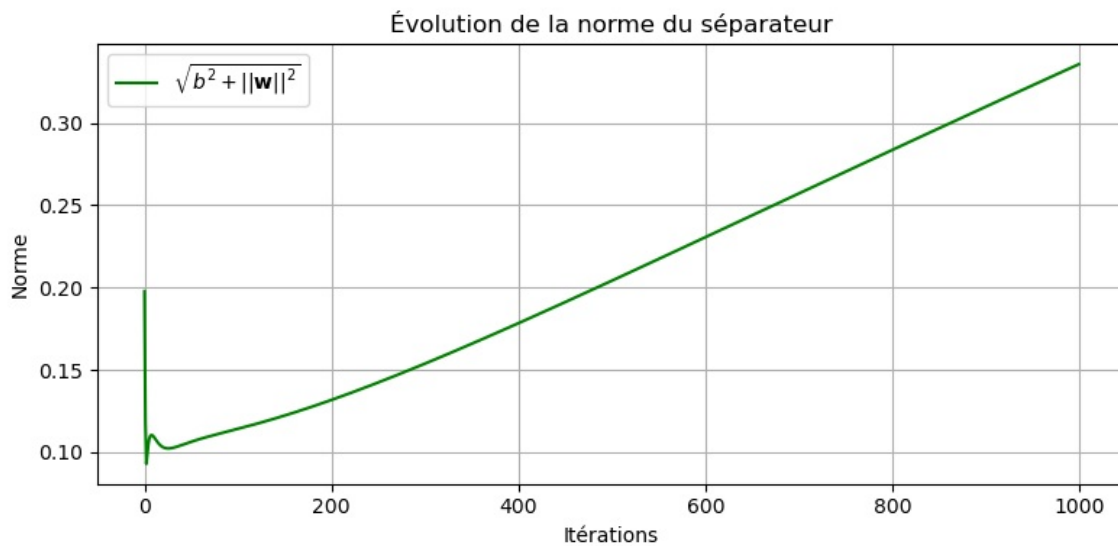
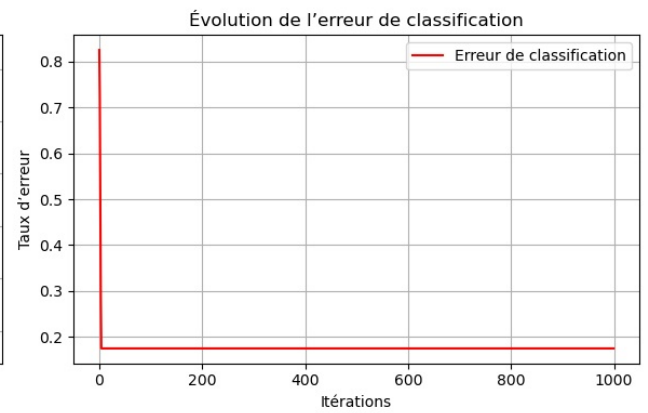
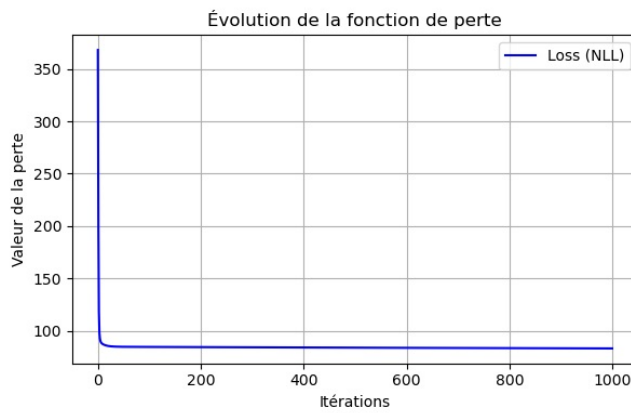
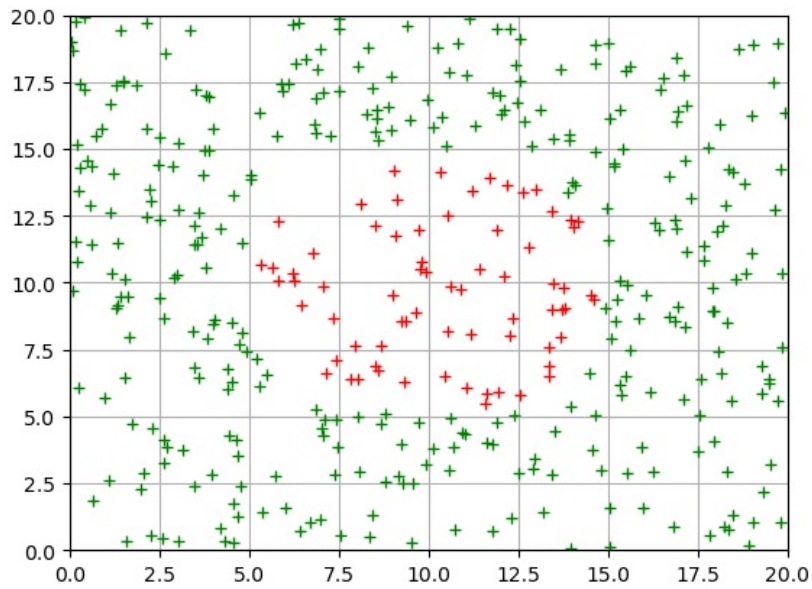
axs[0].plot(losses, label="Loss (NLL)", color='blue')
axs[0].set_title("Évolution de la fonction de perte")
axs[0].set_xlabel("Itérations")
axs[0].set_ylabel("Valeur de la perte")
axs[0].grid()
axs[0].legend()

axs[1].plot(erreur, label="Erreur de classification", color='red')
axs[1].set_title("Évolution de l'erreur de classification")
axs[1].set_xlabel("Itérations")
axs[1].set_ylabel("Taux d'erreur")
axs[1].grid()
axs[1].legend()

plt.tight_layout()
plt.show()

# Figure 4 : norme
plt.figure(figsize=(8, 4))
plt.plot(normes, label=r"$\sqrt{b^2 + ||\mathbf{w}||^2}$", color='green')
plt.title("Évolution de la norme du séparateur")
plt.xlabel("Itérations")
plt.ylabel("Norme")
```

```
plt.grid()
plt.legend()
plt.tight_layout()
plt.show()
```



In []:

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js